TOPIC 3: CONTROL STRUCTURES (SELECTIONS)

At the end of the chapter, the students should be able to:

- *Understand the Boolean expression*
- *Implement decisions using one-way, two-way and multiple way selection structure*
- *Recognize the correct ordering of decisions in multiple branches*
- *Program simple and complex decision*

3.1 CONTROL STRUCTURES

A computer can process a program in one of the following ways:

(i) in sequence
(ii) selectively - by making a choice, which is also called a branch
(iii) repetitively - by executing a statement over and over, using a structure called a loop, or by calling a function.

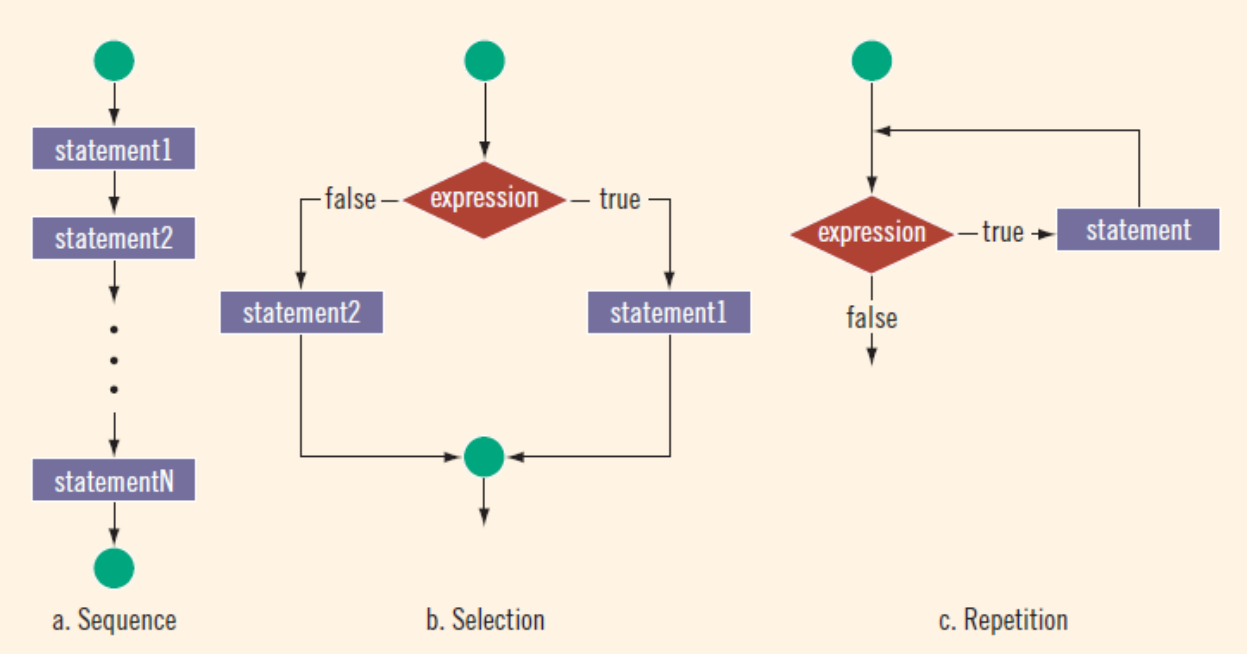

Figure 3.1 Flow of execution

With sequential programs, the computer starts at the beginning and follows the statements in order. No choices are made; there is no repetition. Control structures provide **alternatives** to sequential program execution and are used to **alter the sequential flow of execution**. The two most common control structures are selection and repetition.

In selection, the program executes particular statements depending on some **condition**(s). In repetition, the program repeats particular statements a certain **number of times** based on some condition(s).

```
1. if (score is greater than or equal to 90)
      grade is A
2. if (hours worked are less than or equal to 40)
      wages = rate * hours
   otherwise
      wages = (rate * 40) + 1.5 *(rate *(hours – 40))
3. if (temperature is greater than 70 degrees and it is not
      raining)
      Go golfing!
```

Figure 3.2 Examples of conditional statements

These statements are examples of conditional statements. You can see that certain statements are to be **executed only if certain conditions are met**. A condition is met if it evaluates to true.

3.2 RELATIONAL OPERATORS

A relational operator allows you to make **comparisons** in a program. C++ includes six relational operators that allow you to state conditions and make comparisons.

Operator	Description
==	equal to
!=	not equal to
<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to

Table 3.1 Relational operators in C++

In C++, a condition is represented by a **logical (Boolean) expression**. An expression that has a value of either true or false is called a logical (Boolean) expression. Moreover, true and false are **logical (Boolean) values**.

Example of Boolean expression:

$$i < j$$

where *i* and *j* are variables.

Each of the relational operators is a **binary operator**; that is, it requires two operands. Because the result of a comparison is true or false, expressions using these operators evaluate to *true* or *false*.

3.2.1 Relational Operators with Simple Data Types

You can use the relational operators with all three simple data types.

Expression	Meaning	Value
`8 < 15`	8 is less than 15	`true`
`6 != 6`	6 is not equal to 6	`false`
`2.5 > 5.8`	2.5 is greater than 5.8	`false`
`5.9 <= 7.5`	5.9 is less than or equal to 7.5	`true`

Example 3.1 Expressions using integers and real numbers

For *char* values, whether an expression using relational operators evaluates to true or false depends on the **ASCII collating sequence**.

ASCII Value	Char	ASCII Value	Char	ASCII Value	Char	ASCII Value	Char
32	' '	61	=	81	Q	105	i
33	!	62	>	82	R	106	j
34	"	65	A	83	S	107	k
42	*	66	B	84	T	108	l
43	+	67	C	85	U	109	m
45	-	68	D	86	V	110	n
47	/	69	E	87	W	111	o
48	0	70	F	88	X	112	p
49	1	71	G	89	Y	113	q
50	2	72	H	90	Z	114	r
51	3	73	I	97	a	115	s
52	4	74	J	98	b	116	t
53	5	75	K	99	c	117	u
54	6	76	L	100	d	118	v
55	7	77	M	101	e	119	w
56	8	78	N	102	f	120	x
57	9	79	O	103	g	121	y
60	<	80	P	104	h	122	z

Figure 3.3 The collating sequence of some characters

`'R' > 'T'` is `false`

`'+' < '*'` is `false`

`'A' <= 'a'` is `true`

Example 3.2 Comparing characters

Recall that the data type *bool* has two values, *true* and *false*. In C++, *true* and *false* are reserved words. The identifier *true* is set to 1, and the identifier *false* is set to 0.

3.2.2 Relational Operators with the *String* Type

The relational operators can be applied to variables of type *string*. Variables of type *string* are compared character by character, starting with the first character and using the ASCII collating sequence. The character-by-character comparison continues until either a mismatch is found or the last characters have been compared and are equal.

Suppose that you have the statements:

```
string str1 = "Hello";
string str2 = "Hi";
string str3 = "Air";
string str4 = "Bill";
string str5 = "Big";
```

The following expressions show how string relational expressions evaluate.

Expression	Value /Explanation
`str1 < str2`	`true` `str1 = "Hello"` and `str2 = "Hi"`. The first characters of `str1` and `str2` are the same, but the second character `'e'` of `str1` is less than the second character `'i'` of `str2`. Therefore, `str1 < str2` is `true`.
`str1 > "Hen"`	`false` `str1 = "Hello"`. The first two characters of `str1` and `"Hen"` are the same, but the third character `'l'` of `str1` is less than the third character `'n'` of `"Hen"`. Therefore, `str1 > "Hen"` is `false`.
`str3 < "An"`	`true` `str3 = "Air"`. The first characters of `str3` and `"An"` are the same, but the second character `'i'` of `"Air"` is less than the second character `'n'` of `"An"`. Therefore, `str3 < "An"` is `true`.
`str1 == "hello"`	`false` `str1 = "Hello"`. The first character `'H'` of `str1` is less than the first character `'h'` of `"hello"` because the ASCII value of `'H'` is 72, and the ASCII value of `'h'` is 104. Therefore, `str1 == "hello"` is `false`.
`str3 <= str4`	`true` `str3 = "Air"` and `str4 = "Bill"`. The first character `'A'` of `str3` is less than the first character `'B'` of `str4`. Therefore, `str3 <= str4` is `true`.
`str2 > str4`	`true` `str2 = "Hi"` and `str4 = "Bill"`. The first character `'H'` of `str2` is greater than the first character `'B'` of `str4`. Therefore, `str2 > str4` is `true`.

Expression	Value/Explanation
`str4 >= "Billy"`	`false` `str4 = "Bill"`. It has four characters, and `"Billy"` has five characters. Therefore, `str4` is the shorter string. All four characters of `str4` are the same as the corresponding first four characters of `"Billy"`, and `"Billy"` is the larger string. Therefore, `str4 >= "Billy"` is `false`.
`str5 <= "Bigger"`	`true` `str5 = "Big"`. It has three characters, and `"Bigger"` has six characters. Therefore, `str5` is the shorter string. All three characters of `str5` are the same as the corresponding first three characters of `"Bigger"`, and `"Bigger"` is the larger string. Therefore, `str5 <= "Bigger"` is `true`.

Example 3.3 Comparisons for variables of type *string*

3.3 LOGICAL (BOOLEAN) OPERATORS AND LOGICAL EXPRESSIONS

Logical (Boolean) operators enable you to combine logical expressions. C++ has three logical (Boolean) operators.

Operator	Description
`!`	not
`&&`	and
`||`	or

Table 3.2 Logical (Boolean) operators in C++

Logical operators take only logical values as operands and yield only logical values as results. The operator ! is unary, so it has only one operand. The operators && and || are binary operators.

When you use the ! operator, *!true* is *false* and *!false* is *true*. Putting ! in front of a logical expression reverses the value of that logical expression.

Expression	Value	Explanation
`!('A' > 'B')`	`true`	Because `'A' > 'B'` is `false`, `!('A' > 'B')` is `true`.
`!(6 <= 7)`	`false`	Because `6 <= 7` is `true`, `!(6 <= 7)` is `false`.

Example 3.4 The ! (Not) operator

Expression1	Expression2	Expression1 && Expression2
true (nonzero)	true (nonzero)	true (1)
true (nonzero)	false (0)	false (0)
false (0)	true (nonzero)	false (0)
false (0)	false (0)	false (0)

Table 3.3 The && (And) Operator

Expression	Value	Explanation
(14 >= 5) && ('A' < 'B')	true	Because (14 >= 5) is true, ('A' < 'B') is true, and true && true is true, the expression evaluates to true.
(24 >= 35) && ('A' < 'B')	false	Because (24 >= 35) is false, ('A' <'B') is true, and false && true is false, the expression evaluates to false.

Example 3.5 The && (And) operator

Expression1	Expression2	Expression1 || Expression2
true (nonzero)	true (nonzero)	true (1)
true (nonzero)	false (0)	true (1)
false (0)	true (nonzero)	true (1)
false (0)	false (0)	false (0)

Table 3.4 The || (Or) operator

Expression	Value	Explanation
(14 >= 5) || ('A' > 'B')	true	Because (14 >= 5) is true, ('A' > 'B') is false, and true || false is true, the expression evaluates to true.
(24 >= 35) || ('A' > 'B')	false	Because (24 >= 35) is false, ('A' > 'B') is false, and false || false is false, the expression evaluates to false.
('A' <= 'a') || (7 != 7)	true	Because ('A' <= 'a') is true, (7 != 7) is false, and true || false is true, the expression evaluates to true.

Example 3.6 The || (Or) operator

3.4 SELECTION: if AND if...else

In C++, there are two selections, or **branch control structures**: *if* statements and the *switch* structure. This section discusses how *if* and *if. . .else* statements can be used to create **one-way selection**, **two-way selection**, and **multiple selections**.

3.4.1 One-Way Selection

The syntax of one-way selection is:

```
if (expression)
    statement
```

It begins with the reserved word *if*, followed by an expression contained within parentheses, followed by a statement. Note that the parentheses around the expression are part of the syntax. The expression is sometimes called a **decision maker** because it decides whether to execute the statement that follows it. The expression is usually a logical expression. If the value of the expression is *true*, the statement executes. If the value is *false*, the statement does not execute and the computer goes on to the next statement in the program. The statement following the expression is sometimes called the **action statement**.

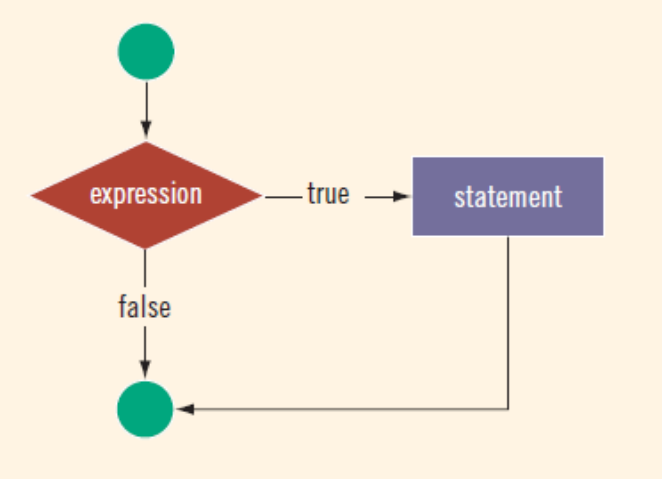

Figure 3.4 The flow of execution of the one –way selection

```
if (score >= 60)
    grade = 'P';
```

In this code, if the expression (`score >= 60`) evaluates to `true`, the assignment statement, `grade = 'P';`, executes. If the expression evaluates to `false`, the statements (if any) following the `if` structure execute. For example, if the value of `score` is `65`, the value assigned to the variable `grade` is `'P'`.

Example 3.7 Flow of execution for one-way selection

```
#include <iostream>

using namespace std;

int main()
{
    int number, temp;

    cout << "Line 1: Enter an integer: ";          //Line 1
    cin >> number;                                 //Line 2
    cout << endl;                                  //Line 3

    temp = number;                                 //Line 4

    if (number < 0)                                //Line 5
        number = -number;                          //Line 6

    cout << "Line 7: The absolute value of "
         << temp << " is " << number << endl;      //Line 7

    return 0;
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 1: Enter an integer: -6734
Line 7: The absolute value of -6734 is 6734
```

Coding 3.1 Sample program for one-way selection

3.4.2 Two-Way Selection

To choose between two alternatives—that is, to implement two-way selections—C++ provides the *if. . .else* statement. Two-way selection uses the following syntax:

```
if (expression)
    statement1
else
    statement2
```

It begins with the reserved word *if*, followed by a logical expression contained within parentheses, followed by a statement, followed by the reserved word *else*, followed by a second statement. Statements 1 and 2 are any valid C++ statements. In a two-way selection, if the value of the expression is *true*, statement1 executes. If the value of the expression is *false*, statement2 executes.

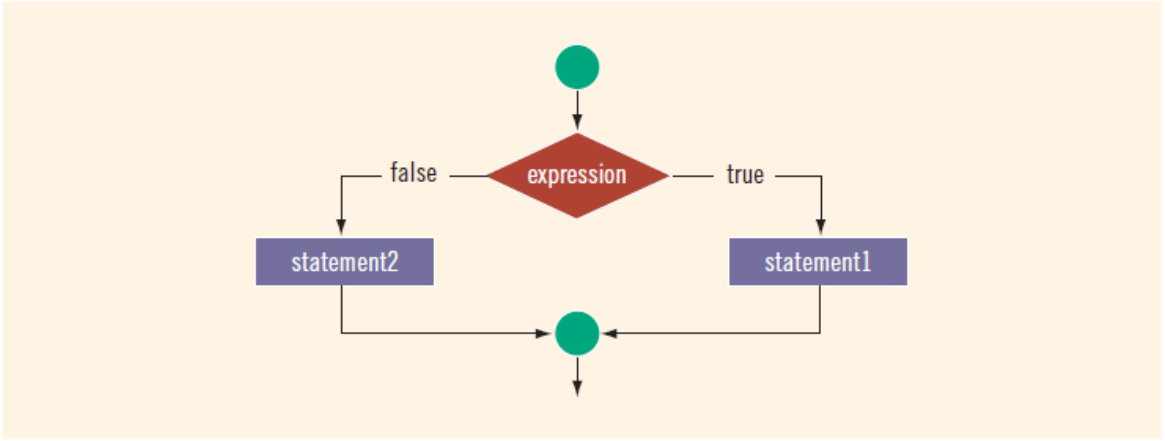

Figure 3.5 Two-way selection

The following program determines an employee's weekly wages. If the hours worked exceed 40, wages include overtime payment.

```
//Program: Weekly wages

#include <iostream>
#include <iomanip>

using namespace std;

int main()

    double wages, rate, hours;

    cout << fixed << showpoint << setprecision(2);     //Line 1
    cout << "Line 2: Enter working hours and rate: "; //Line 2
    cin >> hours >> rate;                             //Line 3

    if (hours > 40.0)                                 //Line 4
        wages = 40.0 * rate +
                1.5 * rate * (hours - 40.0);          //Line 5
    else                                              //Line 6
        wages = hours * rate;                         //Line 7

    cout << endl;                                     //Line 8
    cout << "Line 9: The wages are $" << wages
         << endl;                                     //Line 9

    return 0;
}
```

Sample Run: In this sample run, the user input is shaded.

```
Line 2: Enter working hours and rate: 56.45 12.50

Line 9: The wages are $808.44
```

Coding 3.2 Sample program for two-way selection

3.4.2.1 Compound (Block of) Statements

The *if* and *if. . .else* structures control only one statement at a time. Suppose, however, that you want to execute more than one statement if the expression in an *if* or *if. . .else* statement evaluates to *true*. To permit more complex statements, C++ provides a structure called a **compound statement** or a **block of statements**. A compound statement takes the following form:

```
{
    statement_1
    statement_2
       .
       .
       .
    statement_n
}
```

That is, a compound statement consists of a sequence of statements enclosed in curly braces, { and }. In an *if* or *if . . .else* structure, a compound statement functions as if it was a single statement.

```
if (age >= 18)
    cout << "Eligible to vote." << endl;
else
    cout << "Not eligible to vote." << endl;
```

Coding 3.3 Two-way selection without compound statement

```
if (age >= 18)
{
    cout << "Eligible to vote." << endl;
    cout << "No longer a minor." << endl;
}
else
{
    cout << "Not eligible to vote." << endl;
    cout << "Still a minor." << endl;
}
```

Coding 3.4 Two-way selection with compound statement

3.4.3 Multiple Selections

Suppose that `balance` and `interestRate` are variables of type `double`. The following statements determine the `interestRate` depending on the value of the `balance`.

```
if (balance > 50000.00)                 //Line 1
    interestRate = 0.07;                //Line 2
else                                    //Line 3
    if (balance >= 25000.00)            //Line 4
        interestRate = 0.05;            //Line 5
    else                                //Line 6
        if (balance >= 1000.00)         //Line 7
            interestRate = 0.03;        //Line 8
        else                            //Line 9
            interestRate = 0.00;        //Line 10
```

Coding 3.5 Example of codes with multiple way selection

Assume that score is a variable of type int. Based on the value of score, the following code outputs the grade.

```
if (score >= 90)
    cout << "The grade is A." << endl;
else if (score >= 80)
    cout << "The grade is B." << endl;
else if (score >= 70)
    cout << "The grade is C." << endl;
else if (score >= 60)
    cout << "The grade is D." << endl;
else
    cout << "The grade is F." << endl;
```

Coding 3.6 Example of codes with multiple way selection

3.4.3.1 *switch* Structures

Recall that there are two selection, or branch, structures in C++. The first selection structure, which is implemented with *if* and *if. . .else* statements, usually requires the evaluation of a (logical) expression. The second selection structure, which does not require the evaluation of a logical expression, is called the *switch* structure. C++'s *switch* structure gives the computer the power to choose from among many alternatives.

A general syntax of the *switch* statement is:

```
switch (expression)
{
case value1:
    statements1
    break;
case value2:
    statements2
    break;
    .
    .
    .
case valuen:
    statementsn
    break;
default:
    statements
}
```

In C++, *switch*, *case*, *break*, and *default* are reserved words. In a *switch* structure, first the expression is evaluated. The value of the expression is then used to perform the actions specified in the statements that follow the reserved word case.

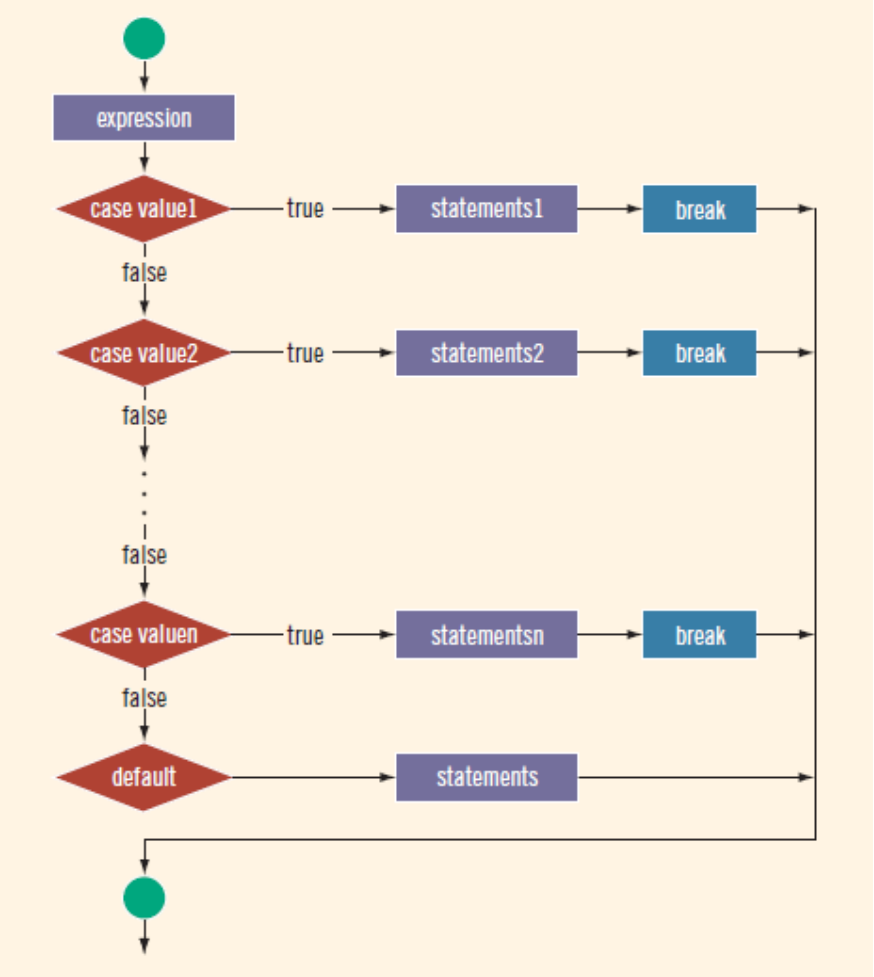

Figure 4.6 *switch* statements

The *switch* statement executes according to the following rules:

1. When the value of the expression is matched against a *case* value (also called a label), the statements execute until either a *break* statement is found or the end of the switch structure is reached.
2. If the value of the expression does not match any of the *case* values, the statements following the *default* label execute. If the *switch* structure has no *default* label and if the value of the expression does not match any of the *case* values, the entire *switch* statement is skipped.
3. A *break* statement causes an immediate exit from the *switch* structure.

Consider the following statements, in which grade is a variable of type char.

```
switch (grade)
{
case 'A':
    cout << "The grade point is 4.0.";
    break;
case 'B':
    cout << "The grade point is 3.0.";
    break;
case 'C':
    cout << "The grade point is 2.0.";
    break;
case 'D':
    cout << "The grade point is 1.0.";
    break;
case 'F':
    cout << "The grade point is 0.0.";
    break;
default:
    cout << "The grade is invalid.";
}
```

Coding 3.7 Example of *switch* statement